



Universidade do Minho
Licenciatura em Engenharia Informática

Unidade Curricular de Bases de Dados

Ano Lectivo de 2025/2026

The Traveler's Notebook

**Ivo Horta, Duarte Arruda, Guilherme Costa, Vasco
Machado**

12/2025

BD

Data de Recepção	
Responsável	
Avaliação	
Observações	

The Traveler's Notebook

Ivo Horta, Duarte Arruda, Guilherme Costa, Vasco
Machado

12/2025

Resumo

Domínio, Senhor Io e o Joe conheceram-se no início da sua vida adulta e aproveitaram os benefícios da sua idade, para desenvolver um projeto que todos concordaram que seria algo muito útil, uma plataforma simples para consulta e partilha de registos de viagens, a qual facilitaria a escolha de lugares a visitar e a evitar. Deste modo, começaram por falar com amigos e familiares sobre as suas experiências e anotar em papel as suas opiniões. De seguida fizeram uma compilação, mas à medida que a palavra se foi espalhando, mais conhecidos, familiares distantes e conhecidos de conhecidos foram dando os seus testemunhos e pedindo para ter acesso a estas informações. Ao se aperceberem das dimensões que este projeto estava a tomar, decidiram investir numa pequena empresa e começar a capitalizar esta atividade que realizavam.

Imediatamente se aperceberam da necessidade de uma BD e da definição do contexto em que esta seria desenvolvida, reunindo-se com a equipa informática que seria responsável por este projeto para a discussão do plano de desenvolvimento da mesma. Após terem definido e concordado com o método de levantamento de requisitos, estes foram levantados e posteriormente organizados. Seguiu-se a construção e documentação de um diagrama Entidade-Relacionamento (ER), que foi convertido num modelo lógico relacional, normalizado até à terceira forma normal (3FN) e posteriormente validado através da implementação de interrogações com recurso a álgebra relacional. Este modelo foi convertido para um modelo físico no Sistema de Gestão de Bases de Dados (SGBD) em MySQL. A BD foi então povoada, foram criados vários utilizadores com diferentes privilégios e foram implementadas as interrogações, alguns procedimentos, funções e gatilhos essenciais para o seu bom funcionamento. Após estar funcional, a BD viu o seu desempenho melhorado através de indexação.

Esta equipa deve agora continuar a prestar os seus serviços à *The Traveler's Notebook*, garantido uma boa operacionalidade da BD e tentando melhorar o seu funcionamento para os seus utilizadores.

Índice

Lista de Siglas e Acrónimos	3
Índice de Figuras	4
Índice de Tabelas	5
1. Definição do sistema	1
1.1. Contextualização	1
1.2. Motivação e Objectivos	1
1.3. Análise da viabilidade do processo	2
1.4. Recursos e equipa de trabalho	3
1.5 Plano de execução do projeto	4
2. Levantamento e Análise de Requisitos	5
2.1. Método de Levantamento e de Análise de Requisitos Adotado	5
2.2. Organização dos Requisitos Levantados	6
3. Modelação Conceptual	11
3.1 Apresentação da abordagem de Modelação	11
3.2. Identificação e Caracterização das Entidades	11
3.3. Identificação e Caracterização dos Relacionamentos	13
3.4. Identificação e Caracterização dos Atributos das Entidades	15
3.5. Apresentação e Explicação do Diagrama ER	18
4. Modelação Lógica	20
4.1. Construção e Validação do Modelo de Dados Lógico	20
4.2. Apresentação e Explicação do Modelo Lógico Produzido	20
4.3. Normalização de Dados	24
4.4. Validação do Modelo com Interrogações do Utilizador	25
5. Implementação Física	29
5.1 Apresentação e Explicação da Base de Dados Implementada	29
5.2 Criação de utilizadores da base de dados	31
5.3 Povoamento da base de dados	34
5.4. Cálculo do espaço da base de dados	35
5.6 Tradução das interrogações do utilizador para SQL	41
5.7 Indexação do Sistema de Dados	43
5.8 Implementação de procedimentos, funções e gatilhos	45
6. Conclusões e Trabalho Futuro	49
Anexos	51

Lista de Siglas e Acrónimos

1FN	Primeira Forma Normal
2FN	Segunda Forma Normal
3FN	Terceira Forma Normal
BD	Base(s) de Dados
CEO	Chief Executive Officer (Diretor Executivo)
ER	Entidade-Relacionamento
FK	Foreign Key (Chave Estrangeira)
ISO	International Organization for Standardization
KB	Kilobyte
LMD	Linguagem de Manipulação de Dados
PK	Primary Key (Chave Primária)
RC	Requisitos de Controlo
RD	Requisitos de Descrição
RGPD	Regulamento Geral sobre a Proteção de Dados
RM	Requisitos de Manipulação
SBD	Sistema de Base de Dados
SGBD	Sistema de Gestão de Bases de Dados
SQL	Structured Query Language (Linguagem de Consulta Estruturada)

Índice de Figuras

- Figura 1 - Diagrama de GANTT
- Figura 2 - Modelo conceptual
- Figura 3 - Modelo lógico
- Figura 4 - Criação da base de dados
- Figura 5 - Tabela de utilizador
- Figura 6 - Tabela cidade
- Figura 7 - Tabela ponto de interesse
- Figura 8 - Tabela avaliação
- Figura 9 - Criação de papéis
- Figura 10 - Permissões de gestor
- Figura 11 - Permissões de moderador
- Figura 12 - Permissões de utilizador
- Figura 13 - Criação de utilizadores
- Figura 14 - Criação de um *Default Role*
- Figura 15 - Povoamento da cidade
- Figura 16 - Povoamento da avaliação
- Figura 17 - Criação de uma vista de moderador
- Figura 18 - Permissões para a vista de moderador
- Figura 19 - *Query 1*
- Figura 20 - *Query 2*
- Figura 21 - *Query 3*
- Figura 22 - *Query 4*
- Figura 23 - *Query 5*
- Figura 24 - Procedure inserir ponto de interesse
- Figura 25 - *Query 6*
- Figura 26 - Atualização das classificações médias

Índice de Tabelas

Tabela 1- Primeira tabela global de requisitos

Tabela 2- Primeiros requisitos de descrição

Tabela 3- Primeiros requisitos de manipulação

Tabela 4- Primeiros requisitos de controlo

Tabela 5- Descrição das entidades

Tabela 6 - Descrição dos relacionamentos

Tabela 7- Descrição da associação dos atributos do utilizador

Tabela 8- Descrição da associação dos atributos da avaliação

Tabela 9- Descrição da associação dos atributos da cidade

Tabela 10- Descrição da associação dos atributos do ponto de interesse

Tabela 11- Símbolos e operadores de álgebra relacional utilizados

Tabela 12- Tamanho em bytes dos tipos de dados utilizados

Tabela 13- Tamanho estimado em bytes de um Utilizador

Tabela 14- Tamanho estimado em bytes de uma Avaliação

Tabela 15- Tamanho estimado em bytes de uma Cidade

Tabela 16- Tamanho estimado em bytes de um Ponto de Interesse

Tabela 17- Tamanho estimado da base de dados

1. Definição do sistema

1.1. Contextualização

Dom Ínio, Senhor Io e o Joe conheceram-se no início da sua vida adulta e aproveitaram os benefícios da sua idade, pelo que perceberam que se houvesse uma plataforma simples para consulta e partilha de registos de viagens ia facilitar a escolha de lugares a visitar ou evitar. Deste modo, começaram por falar com amigos e familiares sobre as suas experiências e anotar em papel e depois fizeram uma compilação, mas à medida que a palavra se foi espalhando mais conhecidos, familiares distantes e conhecidos de conhecidos foram dando os seus testemunhos e pedindo para ter acesso à informação. Toda esta procura levou-os a investir numa pequena empresa e começar a capitalizar esta atividade que realizavam.

1.2. Motivação e Objectivos

Após a criação da *The Traveler 's Notebook*, o interesse das pessoas assim como as recomendações feitas pelos mesmos a amigos e familiares levou a um enorme crescimento deste projeto, uma vez que esta ideia visava resolver um problema muito comum em viagens. Esta excelente reação e adesão das pessoas ao projeto, levou a um aumento exponencial na necessidade de reter, guardar e de organizar a informação relevante para os utilizadores com base nos destinos desejados.

Assim ficou claro que os métodos iniciais para obter, organizar e compartilhar informações já não seriam suficientes. O aumento repentino no número de destinos registados, avaliações compartilhadas e pessoas interessadas trouxe uma complexidade que era difícil de lidar. Cada nova informação precisava ser validada, agrupada corretamente por destino, conectada aos usuários necessários e apresentada de forma consistente na plataforma e tudo isso era cada vez mais demorado e difícil, propenso a erros e complicado de manter atualizado, dificultando assim a viabilidade do projeto.

Surgiu então a necessidade de recorrer a uma base de dados (BD) de modo a resolver estes problemas, melhorando assim a experiência dos utilizadores, aumentando a qualidade e consistência da informação. Assim, esta base de dados permitirá:

- Facilitar a gestão e o registo dos destinos e das viagens, permitindo consultar rapidamente informações como locais e pontos de interesse, reduzindo o risco de haver dados inconsistentes.

- Armazenar de forma estruturada toda a informação importante, como avaliações, fotografias dos locais, opiniões... Tornando mais simples a consulta e análise dos dados, o que permite um aumento na fiabilidade das recomendações dadas aos utilizadores.
- Disponibilizar uma ferramenta essencial para a gestão da plataforma, permitindo acompanhar o crescimento da mesma, organizar os conteúdos partilhados, e otimizar os recursos tanto técnicos como humanos, garantindo assim a escalabilidade do projeto.
- Tornar mais simples a interação com os utilizadores, tornando possível um controlo mais rigoroso dos utilizadores, das publicações e das avaliações evitando erros e melhorando a comunicação entre a plataforma e o utilizador, permitindo que este obtenha mais facilmente a informação desejada.

1.3. Análise da viabilidade do processo

Com a implementação de um sistema de base de dados (SBD), o processo de obtenção, armazenamento e distribuição das informações apresenta muito menos falhas do que utilizando outros métodos, como por exemplo, folhas excel, aumentando assim consideravelmente a produtividade do projeto. Assim, de acordo com a análise feita por uma equipa contratada por Dom Ínio, estima-se que, com esta SBD:

- Os tempos de pesquisa e de procura de informação relativa aos destinos seria 35% menores, devido à maior facilidade de acesso à informação;
- Diminuir em 15% os erros cometidos atualmente com o método utilizado para recolher a informação, evitando inconsistências ou dados desatualizados;
- Um aumento de 8% da precisão das recomendações personalizadas, resultantes da melhoria da organização e da fiabilidade da obtenção dos dados;
- Aumento significativo na segurança e privacidade dos dados dos utilizadores, devido aos mecanismos de controlo de acesso disponibilizados pela mesma;
- Melhorar em 30% a rapidez no carregamento e na disponibilização dos conteúdos, devido as otimizações das consultas e ao armazenamento estruturado;
- Aumentos até 50% da satisfação dos utilizadores consequente da melhoria da experiência de navegação, da rapidez e da qualidade das recomendações.

Depois de obter estes dados, uma reunião entre Dom Ínio, Joe, Senhor Io e a equipa informática, concluíram que a implementação desta base de dados é essencial para uma maior eficácia do projeto assim como uma maior satisfação dos utilizadores.

1.4. Recursos e equipa de trabalho

A equipa necessária para o projeto é dividida em duas categorias, o “pessoal interno” e “pessoal externo”, constituídos estes pelos 3 fundadores do *The Traveler's Notebook*, e pela equipa informática responsável pelo desenvolvimento do SBD. O “pessoal interno” é composto pelo Dom Ínio, o diretor executivo (CEO), principal investidor e a “cara” da Plataforma devido a todos os seus negócios extremamente bem sucedidos, Joe, o diretor de operações de viagens, dono de uma agência de viagens sendo por isso responsável por grande parte da informação obtida relativa aos diferentes destinos, e Senhor Io, diretor de suporte ao utilizador, licenciado em psicologia, motivo pelo qual foi eleito o mais apto para lidar com a parte de suporte e atendimento aos utilizadores. O “pessoal externo” é constituído pela equipa de alunos de Engenharia Informática escolhida por Dom Inio, alunos estes da Universidade do Minho que foram recomendados para este projeto pelo Senhor Io, responsáveis por desenvolver a SBD, implementá-la e testá-la. Os nomes dos integrantes desta equipa são Ivo Horta, Guilherme Costa, Duarte Arruda e Vasco Machado.

Os recursos materiais necessários para este projeto podem dividir-se em duas categorias, software e hardware. Em termos de software, é necessária uma licença para uma aplicação de gestão de projetos, uma aplicação de construção de diagramas ER, para a conceção de modelos lógicos, e um sistema de gestão de base de dados (SGBD). Em termos de hardware, é fundamental existirem computadores para ser usados pelo funcionários da empresa e um servidor onde guardar a BD.

1.5 Plano de execução do projeto

A equipa de alunos reuniu-se com o intuito de planejar o desenvolvimento do SBD. Dessa reunião resultou o seguinte diagrama de Gantt onde se definiu os períodos previstos para a execução de cada parte do processo até à sua conclusão.

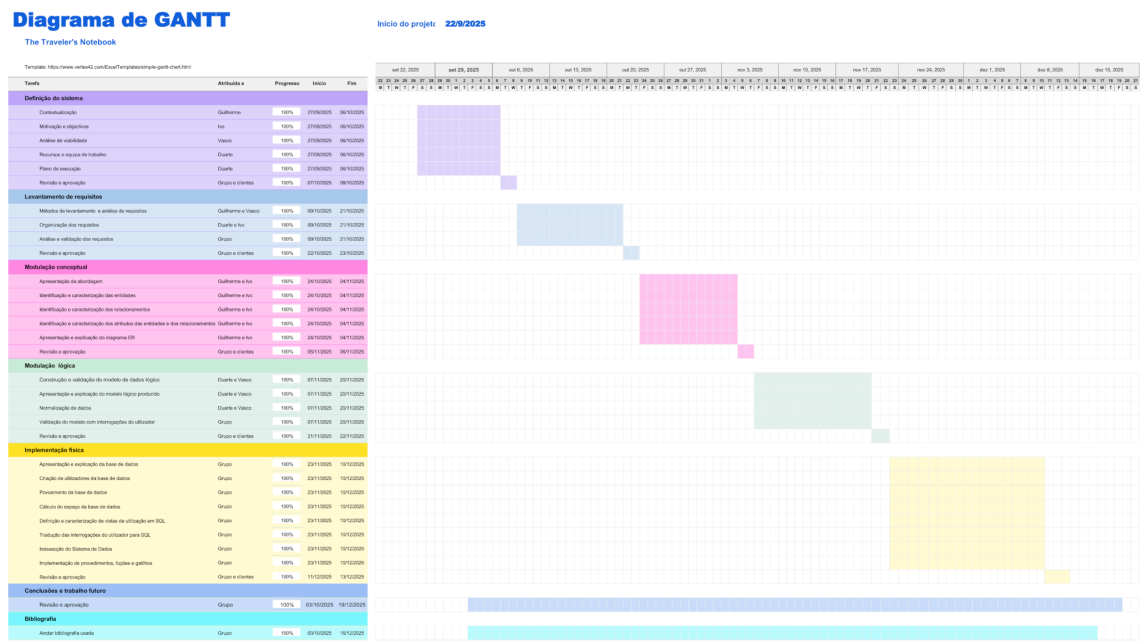


Figura 1 - Diagrama de GANTT

2. Levantamento e Análise de Requisitos

2.1. Método de Levantamento e de Análise de Requisitos

Adotado

Para o bom desenvolvimento da BD e para ter a certeza que tanto o pessoal interno quanto externo responsável pelo projeto estavam na mesma página, a equipa informática decidiu organizar diversas reuniões com os 3 fundadores da *The Traveler's Notebook*.

Foram agendadas então duas reuniões formais com as duas equipas envolvidas neste projeto:

Tabela X- Cabeçalho da ata da primeira reunião de levantamento de requisitos.

Nestas reuniões, a equipa interna detalhou como pretendia que fosse o funcionamento dos processos do projeto:

- como ocorre a comunicação entre os diversos departamentos
- as informações que devem ser registadas e consultadas no sistema
- os níveis de acesso que devem ser atribuídos aos diferentes tipos de colaboradores
- os mecanismos de controle de integridade dos dados, assegurando que não existem viagens sobrepostas ou alocações de recursos indisponíveis.

2.1.1. Reuniões e Dinâmica de Levantamento

As reuniões permitiram um levantamento de requisitos abrangente e coeso. A equipa de informática adotou uma abordagem guiada, harmonizando as diferentes perspetivas: a operacional de conteúdo (Joe), a comercial e estratégica (Dom Ínio) e a de satisfação do utilizador (Senhor Io). Exemplos deste modo de guiar a discussão incluem:

- Sempre que Joe descrevia um novo campo de dados para o catálogo dos destinos (como por exemplo, o país e região) a equipa informática questionava que utilizadores podiam inserir e alterar essa informação, definindo assim os requisitos de segurança e acesso.
- Sempre que Senhor Io mencionava a recolha de dados de utilizadores (como por exemplo, preferências de atividades de lazer), a equipa perguntava qual o prazo de retenção desses mesmos dados e quem deveria ter permissão para os consultar e apagar, fazendo assim referência a requisitos de privacidade (RGPD).

- Foi fundamental conciliar os interesses de Joe de haver flexibilidade na catalogação de novos destinos com a necessidade de Senhor Io de existirem dados padronizados para garantir um suporte eficiente na resolução de dúvidas sobre os locais.
- Foram colocadas questões sobre os valores que os diferentes atributos poderão assumir (como, por exemplo, o valor máximo e mínimo para a avaliação de um destino), de modo a permitir a futura definição dos domínios dos atributos e a criação de *constraints* na BD.

A equipa de informática anotou toda esta informação fornecida, detalhando cada requisito e as regras de negócio associadas às mesmas.

2.1.2. Análise de Documentação Interna

Embora as reuniões tenham sido longas e exaustivas, foi necessário analisar a documentação existente para validar os dados recolhidos e identificar requisitos originados por falhas do sistema manual anterior.

Foi concedido acesso aos documentos digitais da agência de viagens de Joe, que servirá de base de conhecimento para a catalogação inicial da plataforma. A equipa focou-se na análise de *spreadsheets* de destinos e registos de avaliações iniciais em formato excel.

Em particular, o analista responsável procurou por inconsistências ocorridas durante a gestão do destino "Marrocos". O sistema manual falhou ao permitir que o mesmo local tivesse múltiplas entradas com nomes ligeiramente diferentes (ex: "Hotel X" e "Hotel X, Marraquexe"), impedindo o correto agrupamento das avaliações (o problema inicial de Senhor Io e Joe, que acabou por ser o responsável pela ideia deste projeto). Esta análise resultou na formulação de restrições de unicidade e de integridade referencial, garantindo que o novo SBD obrigue a que exista consistência na identificação de cada Ponto de Interesse. A documentação analisada não pode ser divulgada, devido ao acordo de confidencialidade emitido pela agência de Joe .

2.2. Organização dos Requisitos Levantados

Após o levantamento exaustivo nas reuniões com os fundadores e a análise da documentação existente, a equipa informática procedeu à organização e categorização dos requisitos para facilitar as

fases subsequentes de modelação. Os requisitos foram inicialmente classificados em três tipos principais, consoante a sua natureza na gestão da Base de Dados (BD):

- Requisitos de Descrição (RD): Associados à estrutura, entidades e atributos necessários para descrever a informação do negócio.
- Requisitos de Manipulação (RM): Referentes às operações (inserção, consulta, atualização, eliminação) que serão executadas sobre os dados.
- Requisitos de Controlo (RC): Relacionados com a segurança, integridade, privacidade (RGPD) e desempenho do sistema.

A tabela completa de requisitos levantados encontra-se detalhada no Anexo I. Para ilustrar a categorização, apresentam-se em seguida alguns exemplos iniciais de cada tipo:

ID	Descrição Requisito	Tipo	Prioridade	Origem	Analista
1	O sistema deve armazenar o nome e país de cada destino.	RD	Alta	Joe	Duarte Arruda
2	O sistema deve permitir o registo de utilizadores com nome e email.	RD	Alta	Dom Ínio	Duarte Arruda
3	As avaliações são constituídas por ID, data, classificação e um possível comentário.	RD	Alta	Joe	Ivo Horta
4	O sistema deve permitir a consulta de todos os locais visitados (Museus, Parques, Restaurantes) de um dado destino.	RM	Alta	Joe	Ivo Horta
5	O sistema deve permitir que o utilizador insira uma avaliação de 0 a 5 estrelas, fotografias e comentários sobre um local.	RM	Alta	Senhor lo	Duarte Arruda
6	O sistema deve permitir que os utilizadores apaguem os seus dados pessoais segundo o regulamento estabelecido no RGPD.	RM	Crítica	Senhor lo	Ivo Horta
7	O sistema deve garantir que o cálculo da avaliação média de um destino é executado rapidamente (em menos de 2 segundos).	RC	Crítica	Joe	Ivo Horta
8	O sistema deve garantir que apenas utilizadores registados possam fazer reviews.	RC	Crítica	Dom Ínio	Ivo Horta

Tabela 1- Primeira tabela global de requisitos

ID	Descrição	Vista de utilização	Revisor
RD1	Uma cidade é caracterizada pelo seu nome, país e pelas suas classificações. Devendo conter um ID único.	Utilizador	Duarte Arruda
RD2	Um ponto de interesse é identificado pelo ID, devendo conter o seu nome, tipo, classificação média e rua.	Utilizador	Vasco Machado
RD3	Uma avaliação contém um comentário opcional, classificações e a data em que foi escrita. Esta deve também conter um ID.	Utilizador	Vasco Machado
RD4	Um utilizador deve ser identificado pelo nome, apelido, email e um contacto telefónico opcional, tal como pelo seu ID.	Utilizador	Vasco Machado
RD5	Cada avaliação tem que ser referente a uma cidade ou a um ponto de interesse.	Utilizador	Guilherme Costa
RD6	As classificações referentes às avaliações e às cidades devem incluir uma classificação geral e podem incluir classificações opcionais para alojamentos, paisagem, gastronomia e pontos turísticos. Estas estão contidas entre 0 e 5.	Utilizador	Guilherme Costa
RD7	As classificações referentes aos pontos de interesse e às cidades são calculadas através da média de todas as avaliações associadas ao ponto de interesse ou cidade em questão.	Utilizador	Vasco Machado

Tabela 2- Primeiros requisitos de descrição

ID	Descrição	Vista de utilização	Revisor
RM1	Agilizar a inserção de novas cidades e pontos de interesse na BD.	Gestão	Ivo Horta
RM2	Listar todas as Reviews de um determinado Ponto de Interesse, ordenadas pela data mais recente e com a avaliação visível.	Todos os Utilizadores / Gestor de Suporte	Ivo Horta
RM3	Listar todas as Reviews de uma determinada Cidade, ordenadas pela data mais recente e com a avaliação visível.	Todos os Utilizadores / Gestor de Suporte	Ivo Horta
RM4	Permitir que um Utilizador registado apague a sua própria conta e todos os dados associados.	Utilizador/Gestão	Ivo Horta
RM5	O sistema deve permitir a eliminação de avaliações por parte de um moderador.	Gestão	Ivo Horta
RM6	O sistema deve permitir ao utilizador, ao pesquisar uma cidade, ver quais os pontos de interesse com melhores avaliações na mesma.	Todos os Utilizadores	Ivo Horta
RM7	O sistema deve permitir listar as informações das cidades ordenadas pela classificação geral.	Todos os Utilizadores	Guilherme Costa
RM8	O sistema deve permitir listar as informações dos pontos de interesse ordenados pela classificação média.	Todos os Utilizadores	Guilherme Costa
RM9	Todos os dias às 23:59:59 o sistema calcula as novas classificações das cidades e pontos de interesse.	Gestão	Ivo Horta

Tabela 3- Primeiros requisitos de manipulação

ID	Descrição	Vista de utilização	Revisor
RC1	Garantir a unicidade do campo email na tabela de Utilizadores.	Sistema (Global)	Ivo Horta
RC2	Implementar restrições para garantir que a Avaliação de uma Review está sempre entre 0 e 5.	Sistema (Global)	Ivo Horta
RC3	Restringir as permissões de eliminação da tabela de Destinos apenas ao perfil de Gestor de Conteúdos	Gestor de conteúdos	Ivo Horta
RC4	Apenas utilizadores registados podem	Sistema (Global)	Ivo Horta

	inserir novas Reviews e Classificações (0 a 5 estrelas)		
--	---	--	--

Tabela 4- Primeiros requisitos de controlo

3. Modelação Conceptual

3.1 Apresentação da abordagem de Modelação

Para facilitar a visualização e a organização do projeto, a equipa decidiu construir diagrama ER (entidade relacionamento) na plataforma *brModelo*, plataforma esta que permite criar representações visuais da estrutura da base de dados utilizando uma variante da notação *Chen minmax*. Este diagrama é constituído por 4 entidades, cada uma delas com diversos atributos, e 4 relacionamentos. Com esta nova representação dos conteúdos necessários para a sua base de dados, a equipa poderá começar a trabalhar na modulação lógica do projeto.

3.2. Identificação e Caracterização das Entidades

Para a construção do modelo conceptual é necessário criar entidades, estas serão a base da BD e representam um conjunto de objetos com propriedades em comum (atributos) e relacionam-se entre si através de relacionamentos. As quatro entidades em questão são o Utilizador, a Avaliação, a Cidade e o Ponto de Interesse. O “Utilizador” (identificado no requisito RD5) faz referência ao indivíduo que terá contacto direto com a interface da *The Traveler’s Notebook*, este virá a ser essencial para o funcionamento da plataforma pois sem utilizadores não haveria avaliações e, consequentemente, não existiriam classificações. Em seguida foi necessário criar uma entidade referente às avaliações realizadas pelo “Utilizador”, e por esse motivo a equipa achou necessário representar “Avaliação” (identificada no requisito RD3) como uma entidade, entidade esta que virá a ser a base de todas as classificações presentes na plataforma. Uma das entidades sujeitas a avaliação é a “Cidade” (identificada no requisito RD1), que é necessária para guardar as informações que o “Utilizador” procura. A última entidade sujeita a avaliação é o “Ponto de Interesse” (identificado no requisito RD2) representa um local específico procurado por visitantes (normalmente com importância histórica ou apenas um lugar de lazer).

Designação	Descrição	Sinónimos	Requisito
Utilizador	Informações pessoais (Nome, Apelido e contactos) e um identificador.	Consumidor	RD4
Avaliação	Entidade referente a uma opinião do utilizador, deve conter uma data, uma classificação geral e um identificador.	Opinião	RD3
Cidade	Informação de localização (país), nome e classificação.		RD1
Ponto de Interesse	Informação de localização (Rua), nome e classificação.	Ponto Turístico/Ponto de Lazer	RD2

Tabela 5- Descrição das entidades.

3.3. Identificação e Caracterização dos Relacionamentos

Os relacionamentos são essenciais para a lógica do modelo conceptual, sem eles não haveria qualquer ligação entre as entidades referidas anteriormente, estes fazem a ligação entre as entidades e são classificados pela sua cardinalidade e pela participação. A cardinalidade contém dois valores que podem ser 0, 1, ou N e representam o número máximo de aparições de cada entidade (ex.: no relacionamento "Avaliação" "feita por" "Utilizador" cardinalidade é de N:1 pois um utilizador pode fazer N avaliações). Já a participação pode ser total ou parcial (T ou P) e representa a necessidade da participação da entidade, ou seja, se a entidade for obrigatória, a sua participação vai ser total, caso contrário será parcial (ex.: no relacionamento "Avaliação" "feita por" "Utilizador" a participação é T:P, pois para cada avaliação tem de existir um utilizador associado, mas o contrário não se verifica). Para este modelo existem 4 relacionamentos, "Avaliação" "feita por" "Utilizador", "Avaliação" "referente a" "Cidade", "Avaliação" "referente a" "Ponto de Interesse", "Cidade" "contém" "Ponto de Interesse". O relacionamento "Avaliação" "feita por" "Utilizador" faz a ligação entre a "Avaliação" e o seu respetivo "Utilizador", este relacionamento tem cardinalidade N:1, pois um utilizador pode fazer N avaliações e a participação é de T:P, pois para cada avaliação tem de existir um utilizador, mas o contrário não se verifica. Os relacionamentos "Avaliação" "referente a" "Cidade" e "Avaliação" "referente a" "Ponto de Interesse" são muito semelhantes, tendo exatamente a mesma cardinalidade e participação, N:1 e P:P respetivamente, a cardinalidade tem o mesmo motivo da anterior e a participação é P:P porque nenhuma das entidades tem de existir, ambas associam uma avaliação à sua respetiva "Cidade" ou ao seu "Ponto de Interesse". O relacionamento "Cidade" "contém" "Ponto de Interesse", significa que cada ponto de interesse tem de estar associado a uma cidade, daí a sua cardinalidade (1:N) e a sua participação (P:T).

Entidade	Relacionamento	Cardinalidade	Participação	Entidade
Avaliação	feita por	N:1	T:P	Utilizador
Avaliação	referente a	N:1	P:P	Cidade
Avaliação	referente a	N:1	P:P	Ponto de Interesse
Cidade	contém	1:N	P:T	Ponto de Interesse

Tabela 6 - Descrição dos relacionamentos.

3.4. Identificação e Caracterização dos Atributos das Entidades

Cada entidade contém alguns atributos que podem servir para identificar (como é o caso do ID em qualquer uma das entidades) ou para ajudar a caracterizar a entidade em questão, estes podem representar um nome, uma classificação, um contacto etc. Estas informações são cruciais para as futuras *queries* a ser implementadas, pois apresentam um conjunto de informações importantes para que a plataforma funcione e seja útil ao utilizador.

O “Utilizador” contém os atributos “ID”, “Nome”, “Apelido” e “Contactos”, sendo “Contactos” um atributo composto por outros dois atributos, “Email” e “Telemóvel”. Enquanto o “ID” é uma chave para identificar o “Utilizador”, os outros são informações pessoais do mesmo.

Atributo	Tipo	Descrição	Domínio	Opcional	Exemplo
ID	Chave	Identificador do utilizador	INT	N	67
Nome	Simples	Primeiro nome do utilizador	VARCHAR(45)	N	Orlando
Apelido	Simples	Último nome do utilizador	VARCHAR(45)	N	Belo
Contactos -Email -Telemóvel	Composto	Formas de contactar o utilizador -Endereço de email -Telemóvel	 -VARCHAR(100) -VARCHAR(17)	 -N -S	 -orlandobel o.queremo s.vinte@ex emplo.com -945945945

Tabela 7- Descrição da associação dos atributos do utilizador.

A entidade “Avaliação” contém os atributos “ID” para identificação da mesma, “Data”, “Comentário” e classificações como informações que serão usadas para a classificação da “Cidade” ou “Ponto de Interesse” em questão.

Atributo	Tipo	Descrição	Domínio	Opcional	Exemplo
ID	Chave	Identificador da avaliação	INT	N	9999
Data	Simples	Data da avaliação	DATE	N	22/12/2025
Comentário	Simples	Comentário do utilizador	VARCHAR(500)	S	Adorei o restaurante "Belo prato"
Classificações	Composto	Classificações do utilizador (geral, ou mais específicas)	- INT	-N	- 5
-Geral			- INT	-S	- 4
-Alojamento		- Classificação da cidade ou do ponto de interesse	- INT	-S	- 3
-Paisagem			- INT	-S	- 2
-Gastronomia			- INT	-S	- 0
-Pontos turísticos		- Classificação dos alojamentos da cidade			
		- Classificação da paisagem da cidade			
		- Classificação da Gastronomia da cidade			
		- Classificação dos pontos turísticos da cidade			

Tabela 8- Descrição da associação dos atributos da avaliação.

As entidades "Cidade" e "Ponto de Interesse" são muito semelhantes, ambas contém um "ID" como chave primária e um "Nome", as diferenças estão na sua forma de localização (enquanto a "Cidade" tem "País" como atributo, o "Ponto de Interesse" tem "Rua") e na sua forma de classificação, pois a "Cidade" tem um atributo "Classificações" composto por vários tipos de avaliação opcional ("Pontos turísticos", "Gastronomia", "Paisagens" e "Alojamentos") e uma mais geral ("Geral") e o "Ponto de Interesse" tem apenas o atributo "Classificação média".

Atributo	Tipo	Descrição	Domínio	Opcional	Exemplo
ID	Chave	Identificador da cidade	INT	N	9998
País	Simples	País da cidade	VARCHAR(45)	N	Portugal
Nome	Simples	Nome da cidade	VARCHAR(45)	N	Braga
Classificações -Geral -Alojamento -Paisagem -Gastronomia -Pontos turísticos	Composto	Classificações do utilizador (geral, ou mais específicas) - Classificação da cidade - Classificação dos alojamentos da cidade - Classificação da paisagem da cidade - Classificação da Gastronomia da cidade - Classificação dos pontos turísticos da cidade	- DECIMAL(3,2) - DECIMAL(3,2) - DECIMAL(3,2) - DECIMAL(3,2) - DECIMAL(3,2)	-N -S -S -S -S	- 1 - 2 - 3 - 4 - 0

Tabela 9- Descrição da associação dos atributos da cidade.

Atributo	Tipo	Descrição	Domínio	Opcional	Exemplo
ID	Chave	Identificador do ponto de interesse	INT	N	123
Nome	Simples	Nome do ponto de interesse	VARCHAR(45)	N	Museu do Belo
Classificação o média	Simples	Classificação média das avaliações do ponto turístico	DECIMAL(3,2)	N	5
Rua	Simples	Nome da rua do ponto de interesse	VARCHAR(75)	N	Rua Orlando Belo

Tabela 10- Descrição da associação dos atributos do ponto de interesse.

3.5. Apresentação e Explicação do Diagrama ER

Com toda esta informação, a equipa construiu o Diagrama de ER, nele estão contidas todas as entidades na forma de retângulos, e ligados a estes retângulos temos os atributos na forma de circunferências, podendo estas estar vazias caso o atributo seja simples, preenchidas caso seja uma chave e a tracejado caso seja opcional, no caso de serem compostos temos ainda atributos ligados a outro atributo (é o caso das “Classificações” e dos “Contactos”). Após adicionar as entidades e os seus respetivos atributos a equipa adicionou os relacionamentos, estes são representados na forma de losangos e encontram-se sempre ligados a duas entidades, é também importante referir a cardinalidade, que está representada em cada relacionamento próximo às entidades.

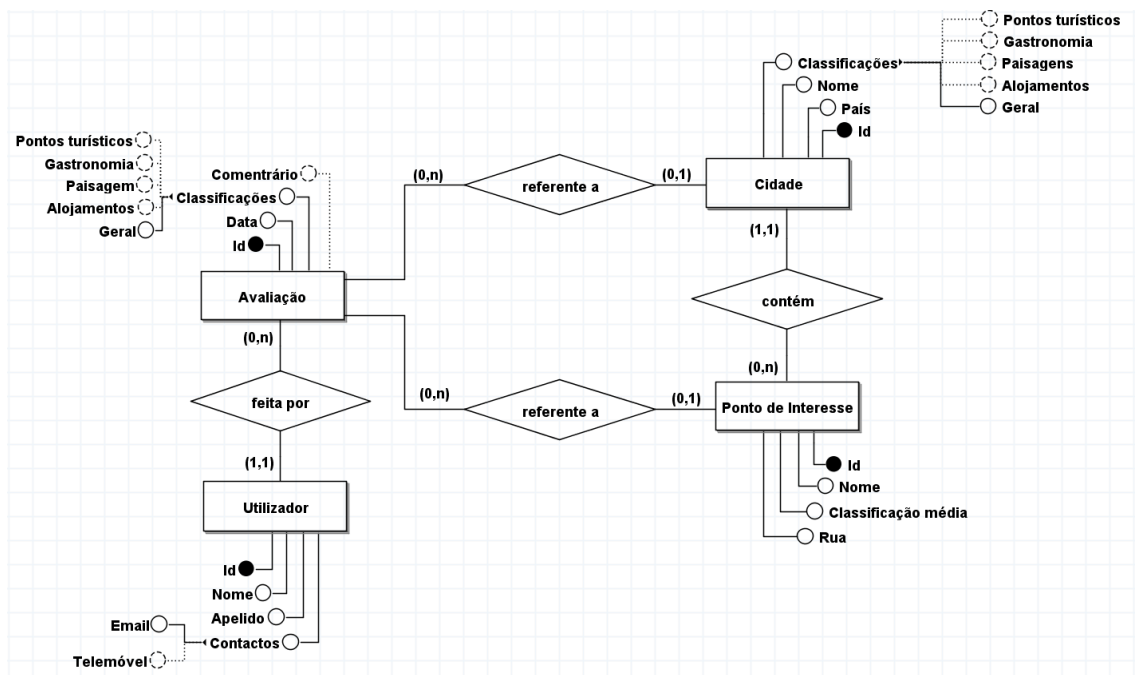


Figura 2 - Modelo conceptual

4. Modelação Lógica

4.1. Construção e Validação do Modelo de Dados Lógico

Nesta fase do projeto, o modelo conceptual já não é suficiente, visto que este ainda não especifica os tipos de dados, as chaves e as restrições que refletem a forma como a base de dados vai funcionar . Por isso, a equipa desenvolveu um modelo lógico com auxílio ferramenta MySQL, com base no modelo conceptual anteriormente definido.

O modelo lógico resulta do refinamento do modelo conceptual, este passa a incluir elementos como as tabelas que representam as entidades, os atributos de cada entidade com os seu tipos devidamente identificados (INT,VARCHAR,DATE) , as chaves primárias (PK) são usadas para identificar unicamente cada registro feito e em conjunto com chaves estrangeiras (FK) vão ser usadas para a estabelecer relações entre tabelas, desta forma a PK vai garantir integridade de identidade e a FK vai garantir integridade referencial . A equipa após a construção do modelo lógico realizou a validação do mesmo de forma a garantir que cada entidade tinha uma PK , as relações do modelo conceptual foram corretamente interpretadas e implementadas sob a forma de Fks, se os dados obrigatórios e opcionais foram devidamente designados por NOT NULL em caso de ser obrigatórios e NOT NULL/NULL se forem opcionais e se o modelo feito é viável e corresponde às necessidades do sistema, que neste caso vai corresponder à avaliação das cidades e pontos de interesse que é realizada utilizadores.

4.2. Apresentação e Explicação do Modelo Lógico Produzido

Após a conclusão do modelo lógico, este ficou composto por quatro tabelas: Utilizador, Cidade, Ponto de Interesse e Avaliação.

Tabela Utilizador

A tabela Utilizador representa os utilizadores do sistema e contém os seguintes atributos:

- ID_Utilizador (INT , PK , NOT NULL)
- Nome (VARCHAR(45) , NOT NULL)
- Apelido (VARCHAR(45) , NOT NULL)
- Email (VARCHAR(45) , NOT NULL)
- Telemóvel (VARCHAR(17) , NULL)

A chave primária ID_Utilizador, permite identificar unicamente cada utilizador, desta forma vai ser possível relacionar a entidade utilizador com a entidade avaliação . Um utilizador é composto pelos

atributos: Nome, Apelido , Email e Telemóvel, sendo que Nome, Apelido e Email são essenciais para caracterizar o utilizador, logo tem carácter obrigatório ,enquanto o atributo Telemóvel é opcional.

Tabela Cidade

A tabela Cidade representa as cidades que podem ser avaliadas e contém os seguintes atributos:

- ID_Cidade (INT , PK , NOT NULL)
- Nome (VARCHAR(45) , NOT NULL)
- Pais (VARCHAR(45) , NOT NULL)
- Classificação_Geral (Decimal(3,2) , NOT NULL)
- Classificação_Alojamentos (Decimal(3,2) , NULL)
- Classificação_Paisagens (Decimal(3,2) , NULL)
- Classificação_Gastronomia (Decimal(3,2) , NULL)
- Classificação_Pontos_turísticos (Decimal(3,2) , NULL)

A chave primária ID_Cidade, é usada para identificar unicamente cada cidade e permite referenciar a entidade Cidade com a tabela Ponto_de_Interesse e a tabela Avaliação. Os atributos Nome e País são obrigatórios . Em seguida, existem atributos de classificação, nomeadamente Classificação_Geral que tem carácter obrigatório sendo os restantes atributos deste campo de carácter opcional nomeadamente Classificação_Alojamentos, Classificação_Paisagens, Classificação_Gastronomia e Classificação_Pontos_turísticos.

Tabela Ponto_de_Interesse

A tabela Ponto_de_Interesse representa os pontos de interesse associados às cidades e contém os seguintes atributos:

- ID_Ponto_de_Interesse (INT , PK , NOT NULL)
- Nome (VARCHAR (45) , NOT NULL)
- Classificação_Média (Decimal(3,2) , NOT NULL)
- Rua (VARCHAR (75) , NOT NULL)
- ID_Cidade (INT , NOT NULL)

A chave primária ID_Ponto_de_Interesse, é usada para identificar unicamente cada Ponto_de_Interesse e permite referenciar a entidade Ponto_de_Interesse na tabela Avaliação. Os atributos Nome, Rua e Classificação_Média são obrigatórios. Além destes atributos essenciais inclui também uma chave estrangeira ID_Cidade, que referencia a tabela Cidade, desta forma permite associar cada ponto de interesse a uma cidade específica, esta relação é do tipo 1:N, pois uma cidade pode conter vários pontos de interesse, mas cada ponto de interesse pertence apenas a uma cidade.

Tabela Avaliação

A tabela Avaliação representa as avaliações efetuadas pelos utilizadores e contém os seguintes atributos:

- ID_Avaliação (INT , PK , NOT NULL)
- Data (Date, NOT NULL)
- Comentário (VARCHAR(500) , NULL)
- Classificação_Geral (Decimal(3,2) , NOT NULL)
- Classificação_Alojamentos (Decimal(3,2) , NULL)
- Classificação_Paisagens (Decimal(3,2) , NULL)
- Classificação_Gastronomia (Decimal(3,2), NULL)
- Classificação_Pontos_turísticos (Decimal(3,2) , NULL)
- ID_Utilizador (INT , NOT NULL)
- ID_Cidade (INT , NOT NULL)
- ID_Ponto_de_Interesse (INT , NOT NULL)

A chave primária ID_Avaliação é usada para identificar unicamente cada avaliação, esta tabela inclui ainda as chaves estrangeiras (FK): ID_Utilizador, ID_Cidade e ID_Ponto_de_Interesse, que permite relacionar o utilizador , a cidade e ponto de interesse. Assim, a tabela Avaliação encontra-se relacionada com as tabelas Utilizador, Cidade e Ponto_de_Interesse através de relações 1:N, uma vez que um utilizador pode realizar várias avaliações, e que uma cidade e um ponto de interesse podem possuir várias avaliações associadas, sendo que cada avaliação pertence a um único utilizador e referência os elementos avaliados através das respetivas chaves estrangeiras ,desta forma garante integridade referencial .

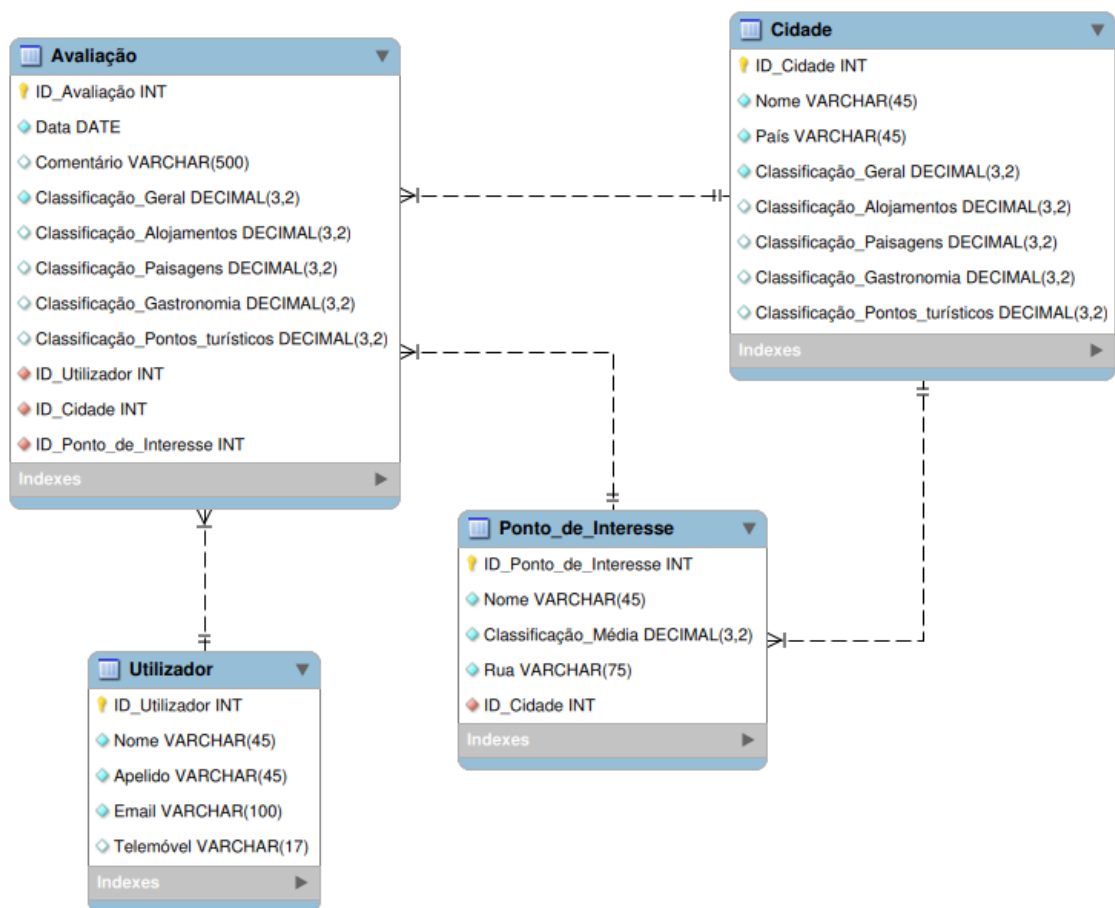


Figura 3 - Modelo lógico

4.3. Normalização de Dados

Após a definição do modelo lógico, a equipa teve em conta os princípios de normalização, com o objetivo de reduzir repetições, evitar dados inválidos e prevenir anomalias de inserção, atualização e remoção. Para isso, foi feita a separação da informação por entidades distintas, o que vai garantir que cada tabela guarda apenas dados diretamente relacionados com a entidade que representa e as relações entre entidades são feitas através de chaves estrangeiras.

De forma geral, o modelo cumpre os requisitos das principais formas normais, o modelo cumpre a regra 1FN visto que os atributos são atômicos, uma vez que cada campo guarda apenas um valor e não existem listas ou estruturas repetidas dentro do mesmo atributo, cumpre os requisitos de 2FN, porque as tabelas utilizam chaves primárias simples e os atributos dependem da totalidade da chave, logo não existe dependências parciais e cumpre os requisitos 3FN porque os atributos que não são chaves descrevem a entidade e dependem apenas da respetiva chave primária de forma a evitar dependências transitivas dentro da mesma tabela.

Ao separar as entidades Utilizador, Cidade, Ponto_de_Interesse e Avaliação em tabelas diferentes, estamos a contribuir para a normalização do sistema, pois permite que os dados de cada entidade sejam armazenados apenas uma vez e reutilizados através da utilização da chave primária. Por sua vez, as avaliações como são armazenadas numa tabela própria e usam as chaves estrangeiras como referência às entidades relevantes, desta forma garante a consistência dos dados e facilita consultas sem necessidade de duplicar informação.

4.4. Validação do Modelo com Interrogações do Utilizador

Para a criação de queries a equipa usou álgebra relacional para as poder representar antes de converter para a linguagem de *SQL*, esta representação consiste na utilização de símbolos que indicam instruções, em seguida está uma tabela com os símbolos que a equipa utilizou para as *queries*:

Símbolo	Significado
τ	Ordenação
π	Projeção
$\bowtie$	Junção
σ	Seleção
$\leftarrow$	Atribuição

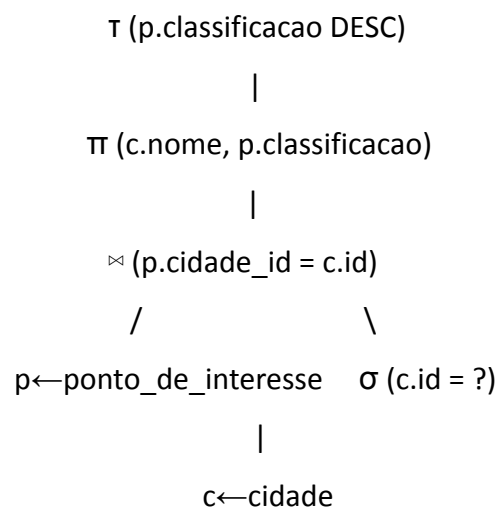
Tabela 11- símbolos e operadores de álgebra relacional utilizados

A primeira *query* mostra uma lista dos pontos de interesse (nome e classificação) de uma cidade escolhida pelo utilizador ordenados pela sua classificação.

Equação:

$$\tau_{p.classificacao \text{ DESC}}(\pi_{p.nome, p.classificacao}(p \leftarrow \text{ponto_de_interesse} \bowtie_{p.cidade_id = c.id} \sigma_{c.id=?}(c \leftarrow \text{cidade})))$$

Árvore de resolução:

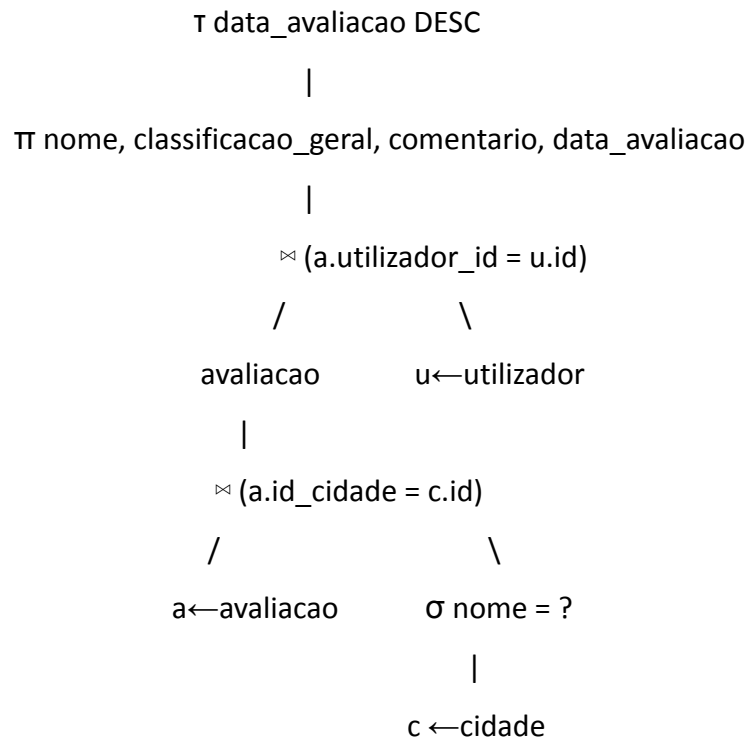


A segunda *query* mostra todos os comentários de uma cidade selecionada pelo utilizador.

Equação:

$$T_{a.data_avaliacao} \text{ DESC } (\pi_{u.nome, a.classificacao_geral, a.comentario, a.data_avaliacao} ((a \leftarrow avaliacao \bowtie_{a.utilizador_id=u.id} u \leftarrow utilizador) \bowtie_{a.id_cidade=c.id} (\sigma_{c.nome=?} (c \leftarrow cidade))))$$

Árvore de resolução:

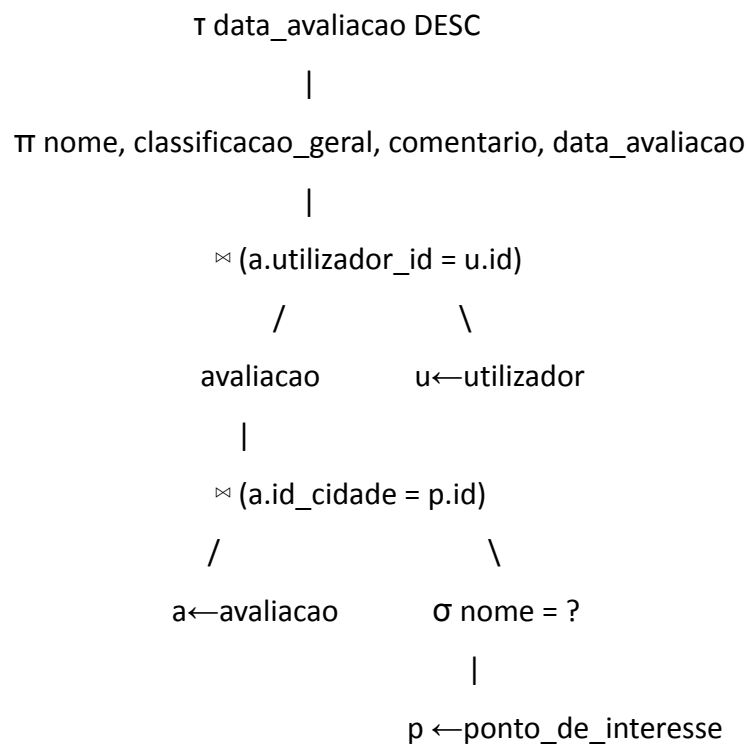


A terceira *query* é muito semelhante à segunda, sendo a única diferença que lista os comentários dos pontos de interesse e não as cidades

Equação:

$$T_{a.data_avaliacao \text{ DESC} (\pi_{u.nome, a.classificacao_geral, a.comentario, a.data_avaliacao} ((a \leftarrow avaliacao) \bowtie_{a.utilizador_id=u.id} u \leftarrow utilizador)) \bowtie_{a.id_ponto_de_interesse=p.id} (\sigma_{p.nome=?} (p \leftarrow ponto_de_interesse)))$$

Árvore de resolução:



A quarta *query* lista as cidades por ordem decrescente de classificação geral.

Equação:

$$T_{\text{classificacao_gera DESC}}(\pi_{\text{nome, pais, classificacao_geral}}(\text{cidade}))$$

Árvore de resolução:

```

      T classificacao_geral DESC
      |
      Π nome, pais, classificacao_geral
      |
      cidade

```

A quinta *query* é muito semelhante à quarta, mas em vez de listar as cidades com melhor classificação, esta lista os pontos de interesse com melhor classificação.

Equação:

$T_{\text{classificacao_gera DESC}}(\Pi_{\text{nome, rua, classificacao_media}}(\text{ponto_de_interesse}))$

Árvore de resolução:

```

      T classificacao_geral DESC
      |
      Π nome, rua, classificacao_media
      |
      ponto de interesse

```

5. Implementação Física

5.1 Apresentação e Explicação da Base de Dados Implementada

O modelo físico da BD é definido por um *script* SQL (*Structured Query Language*) criado pela equipa informática. Este é lido pelo SGBD MySQL que cria, de acordo com as instruções no script, a BD para o projeto e todas as suas tabelas.

O primeiro passo para a escrita deste script foi a criação da estrutura base, feita através das seguintes instruções:

```
CREATE DATABASE trabalhopratico;  
USE trabalhopratico;
```

Figura 4 - Criação da base de dados

Neste excerto observam-se três instruções fundamentais. A primeira, `DROP DATABASE IF EXISTS`, assegura que, caso a base de dados já exista de uma execução anterior, esta seja eliminada. Isto garante um ambiente determinista para testes, permitindo recriar a estrutura do zero sem conflitos com dados antigos ou estruturas obsoletas. De seguida, a instrução `CREATE DATABASE` inicializa a nova base de dados denominada "trabalhopratico". Por fim, a diretiva `USE` informa o MySQL de que as instruções seguintes de criação de tabelas devem ser aplicadas especificamente dentro deste contexto.

Antes de iniciar a criação das tabelas, foi necessário analisar as dependências de chaves estrangeiras. Para garantir a integridade referencial e evitar erros de execução, uma tabela que contenha uma chave estrangeira (tabela filha) só pode ser criada após a existência da tabela que esta referência (tabela pai). Seguindo esta ordem, a equipa começou pela criação da tabela "utilizador", através da seguinte instrução:

```

• CREATE TABLE utilizador (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(45) NOT NULL,
    apelido VARCHAR(45) NOT NULL,
    email VARCHAR(100) UNIQUE,
    telemovel VARCHAR(17)
);

```

Figura 5 - Tabela de utilizador

Nesta relação, destaca-se o uso da propriedade AUTO_INCREMENT no atributo "id". Deste modo, o identificador do utilizador é gerado sequencialmente pelo SGBD, dispensando a inserção manual e garantindo a unicidade da Chave Primária (PRIMARY KEY). Para os campos textuais "nome" e "apelido", utilizou-se o tipo VARCHAR(100) com a restrição NOT NULL, assegurando que todos os utilizadores registados têm a sua identificação pessoal completa. Adicionalmente, aplicou-se a restrição UNIQUE ao campo "email", o que impede ao nível da base de dados que dois utilizadores distintos se registem com o mesmo endereço eletrónico, garantindo a integridade dos dados de contacto.

De seguida, procedeu-se à criação da tabela "cidade", que não depende de quaisquer chaves estrangeiras:

```

• CREATE TABLE cidade (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(45) NOT NULL,
    pais VARCHAR(45) NOT NULL,
    classificacao_geral DECIMAL(3, 2),
    classificacao_pontos_turisticos DECIMAL(3, 2),
    classificacao_gastronomia DECIMAL(3, 2),
    classificacao_paisagens DECIMAL(3, 2),
    classificacao_alojamentos DECIMAL(3, 2)
);

```

Figura 6 - Tabela cidade

Para além da estrutura padrão de identificação, esta tabela armazena as médias das classificações atribuídas à cidade. A equipa optou pelo tipo de dados DECIMAL(3, 1) para estes campos. Esta escolha

permite armazenar valores numéricos com uma casa decimal (ex: 4.5), o que é ideal para médias de avaliações, oferecendo uma precisão superior ao tipo inteiro (INT) e evitando os problemas de arredondamento de vírgula flutuante típicos do tipo FLOAT.

Posteriormente, foi criada a tabela "ponto_de_interesse", que estabelece a primeira relação entre entidades:



```
CREATE TABLE ponto_de_interesse (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(45) NOT NULL,  
    rua VARCHAR(75),  
    classificacao_media DECIMAL(3, 2),  
    cidade_id INT NOT NULL,  
    FOREIGN KEY (cidade_id) REFERENCES cidade(id)  
);
```

Figura 7 - Tabela ponto de interesse

Nesta tabela, observa-se a presença de uma chave estrangeira, definida pela instrução FOREIGN KEY (cidade_id) REFERENCES cidade(id). Esta restrição de integridade referencial obriga a que cada ponto de interesse esteja associado a uma cidade existente na base de dados. O atributo foi definido como NOT NULL, impedindo a existência de pontos turísticos sem localização geográfica definida.

Por último, foi criada a tabela "avaliação", que junta todas as interações do sistema e apresenta a maior complexidade de restrições:

```

CREATE TABLE avaliacao(
    id INT PRIMARY KEY AUTO_INCREMENT,
    data_avaliacao DATETIME DEFAULT CURRENT_TIMESTAMP,
    comentario VARCHAR(500),
    nota_geral INT NOT NULL CHECK (nota_geral BETWEEN 0 AND 5),
    nota_pontos_turisticos INT CHECK (nota_pontos_turisticos BETWEEN 0 AND 5),
    nota_gastronomia INT CHECK (nota_gastronomia BETWEEN 0 AND 5),
    nota_paisagens INT CHECK (nota_paisagens BETWEEN 0 AND 5),
    nota_alojamentos INT CHECK (nota_alojamentos BETWEEN 0 AND 5),
    utilizador_id INT NOT NULL,
    cidade_id INT,
    ponto_interesse_id INT,
    FOREIGN KEY (utilizador_id) REFERENCES utilizador(id) ON DELETE CASCADE,
    FOREIGN KEY (cidade_id) REFERENCES cidade(id),
    FOREIGN KEY (ponto_interesse_id) REFERENCES ponto_de_interesse(id),
    CONSTRAINT chk_origem_avaliacao CHECK (cidade_id IS NOT NULL OR ponto_interesse_id IS NOT NULL)
);

```

Figura 8 - Tabela avaliação

A tabela "avaliacao" apresenta diversas características técnicas relevantes. O atributo "data_avaliacao" possui a propriedade DEFAULT CURRENT_TIMESTAMP, que preenche automaticamente a data e hora do sistema no momento da inserção do registro, caso esta não seja especificada manualmente.

Para garantir a qualidade e consistência dos dados, foram implementadas restrições de domínio CHECK em todas as colunas referentes a notas (como visível em CHECK (nota_geral BETWEEN 1 AND 5)). Isto assegura que o motor de base de dados rejeita qualquer tentativa de inserir valores fora da escala permitida (1 a 5), validando as regras de negócio diretamente no esquema físico.

Finalmente, a tabela contém três chaves estrangeiras. A referência a utilizador_id é obrigatória (NOT NULL), pois toda a avaliação deve ter um autor identificado. Contudo, as referências a cidade_id e ponto_interesse_id não possuem a restrição NOT NULL. Esta decisão de design suporta a lógica de negócio onde uma avaliação pode ser direcionada especificamente a uma cidade ou a um ponto turístico, sem a obrigatoriedade técnica de preencher ambos os campos simultaneamente.

5.2 Criação de utilizadores da base de dados

A Base de Dados "trabalhopratco" será utilizada por vários intervenientes com objetivos e permissões distintas, o que exige um sistema robusto de controle dos acessos. A análise dos requisitos de controle e manipulação levou a equipa de desenvolvimento a optar por uma estratégia de controle baseada em papéis (*Role-Based Access Control*). Assim, em vez de se atribuírem privilégios individualmente a cada funcionário ou cliente, criaram-se *roles* (papéis), que são conjuntos de privilégios que podem ser associados a cada utilizador. A leitura dos requisitos evidencia a existência de três tipos de perfil, que darão origem a três papéis: gestor de conteúdos, moderador e utilizador (cliente).

A criação destes três papéis é feita através das seguintes instruções SQL, que garantem a inicialização do sistema de segurança:

- `CREATE ROLE 'papel_gestor', 'papel_moderador', 'papel_utilizador';`

Figura 9 - Criação de papéis

Tal como ocorre na criação da BD, a primeira instrução, `DROP ROLE IF EXISTS`, elimina os papéis caso já tenham sido criados previamente. Isto é fundamental para garantir um ambiente de desenvolvimento determinista, onde as permissões no MySQL correspondem sempre à descrição providenciada pela versão mais recente do *script*, sem erros de duplicação. De seguida, através da diretiva `CREATE ROLE`, criam-se os papéis.

Para a definição de privilégios, a equipa optou por uma abordagem que privilegia o uso de Procedimentos Armazenados (*Stored Procedures*) em detrimento do acesso direto de escrita nas tabelas. Deste modo, garante-se que as regras de negócio (como verificar se uma cidade existe antes de criar um ponto turístico) são sempre cumpridas.

O primeiro perfil configurado foi o do Gestor de Conteúdos (`papel_gestor`), responsável pelos requisitos RM1 e RC3. A este papel foi concedido controlo total sobre as tabelas de infraestrutura (“cidade” e “ponto_de_interesse”) e permissão de leitura sobre utilizadores e avaliações. Crucialmente, a permissão de inserção de dados é gerida através de procedimentos específicos, conforme as instruções abaixo:

```
-- 2. Permissões GESTOR
GRANT SELECT, INSERT, UPDATE, DELETE ON trabalhopratico.cidade TO 'papel_gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON trabalhopratico.ponto_de_interesse TO 'papel_gestor';
GRANT SELECT ON trabalhopratico.utilizador TO 'papel_gestor';
GRANT SELECT ON trabalhopratico.avaliacao TO 'papel_gestor';
-- Apenas damos permissão para EXECUTAR
GRANT EXECUTE ON PROCEDURE trabalhopratico.InserirCidade TO 'papel_gestor';
GRANT EXECUTE ON PROCEDURE trabalhopratico.InserirPontoInteresse TO 'papel_gestor';
GRANT SELECT ON v_gestor_estatistica_cidades TO 'papel_gestor';
GRANT SELECT ON v_gestor_pontos_criticos TO 'papel_gestor';
GRANT SELECT ON v_moderador_feed_completo TO 'papel_gestor';
```

Figura 10 - Permissões de gestor

De seguida, configurou-se o perfil de Moderador (`papel_moderador`), associado ao requisito RM4. Este utilizador necessita de visualizar todo o conteúdo da base de dados para analisar denúncias, pelo que lhe foi atribuída a permissão `SELECT` global. No entanto, o seu poder de escrita restringe-se à eliminação de avaliações impróprias, operacionalizada através da tabela “avaliacao” e do procedimento respectivo:

```
-- 3. Permissões MODERADOR
GRANT SELECT ON trabalhopratico.* TO 'papel_moderador';
GRANT DELETE ON trabalhopratico.avaliacao TO 'papel_moderador';
GRANT EXECUTE ON PROCEDURE trabalhopratico.ApagarAvaliacao TO 'papel_moderador';
GRANT SELECT ON v_moderador_feed_completo TO 'papel_moderador';
GRANT SELECT ON v_moderador_alertas TO 'papel_moderador';
```

Figura 11 - Permissões de moderador

Por fim, definiram-se as permissões para o Utilizador Registado (papel_utilizador). Este papel, que corresponde aos clientes da plataforma (requisitos RC4, RM2, RM3), necessita de ler informações sobre destinos e inserir as suas próprias avaliações. Para além do acesso de leitura às tabelas principais e de inserção na tabela de avaliações, o utilizador interage com o sistema maioritariamente através de procedimentos que encapsulam lógica complexa, como a geração de rankings ou a eliminação da própria conta:

```
-- 4. Permissões UTILIZADOR
GRANT SELECT ON trabalhopratico.cidade TO 'papel_utilizador';
GRANT SELECT ON trabalhopratico.ponto_de_interesse TO 'papel_utilizador';
GRANT SELECT ON trabalhopratico.avaliacao TO 'papel_utilizador';
GRANT SELECT ON trabalhopratico.utilizador TO 'papel_utilizador';
GRANT INSERT ON trabalhopratico.avaliacao TO 'papel_utilizador';
GRANT EXECUTE ON PROCEDURE trabalhopratico.RankingPontosPorCidade TO 'papel_utilizador';
GRANT EXECUTE ON PROCEDURE trabalhopratico.ListarReviewsDestino TO 'papel_utilizador';
GRANT EXECUTE ON PROCEDURE trabalhopratico.ApagarUtilizador TO 'papel_utilizador';
GRANT EXECUTE ON PROCEDURE trabalhopratico.CriarUtilizador TO 'papel_utilizador';
GRANT SELECT ON v_utilizador_top_pontos TO 'papel_utilizador';
GRANT SELECT ON v_utilizador_reviews_recents TO 'papel_utilizador';
```

Figura 12 - Permissões de utilizador

Após a definição das permissões necessárias, a equipa de desenvolvimento criou três utilizadores de exemplo, um para cada papel. Tal como foi feito anteriormente, utiliza-se a instrução DROP USER IF EXISTS para limpeza do ambiente, seguida da criação das contas com palavras-passe definidas:

```
DROP USER IF EXISTS 'gestor_ivo'@'localhost';
CREATE USER 'gestor_ivo'@'localhost' IDENTIFIED BY 'senha_forte_gestor';
GRANT 'papel_gestor' TO 'gestor_ivo'@'localhost';

DROP USER IF EXISTS 'mod_vasco'@'localhost';
CREATE USER 'mod_vasco'@'localhost' IDENTIFIED BY 'senha_moderador';
GRANT 'papel_moderador' TO 'mod_vasco'@'localhost';

DROP USER IF EXISTS 'user_joao'@'localhost';
CREATE USER 'user_joao'@'localhost' IDENTIFIED BY 'senha_cliente';
GRANT 'papel_utilizador' TO 'user_joao'@'localhost';
```

Figura 13 - Criação de utilizadores

Para concluir a configuração, atribui-se o papel correspondente a cada utilizador como o seu padrão, utilizando a diretiva SET DEFAULT ROLE. Isto garante que os privilégios estão ativos assim que o utilizador inicia a sessão:

```
SET DEFAULT ROLE ALL TO 'gestor_ivo'@'localhost', 'mod_vasco'@'localhost', 'user_joao'@'localhost';
```

Figura 14 - Criação de um *Default Role*

Assim se conclui a criação dos utilizadores da BD, assegurando que cada interveniente possui apenas as permissões estritamente necessárias para o desempenho das suas funções.

5.3 Povoamento da base de dados

O povoamento da Base de Dados (BD) é uma etapa crítica que exige um cumprimento rigoroso das restrições de integridade definidas no modelo físico. Independentemente do volume de dados a inserir, um dos detalhes fundamentais a ter em consideração é a ordem cronológica e lógica pela qual todas as relações são populadas.

Na BD desenvolvida para este projeto, não é possível, por exemplo, inserir registos na tabela “avaliacao” antes de existirem registos nas tabelas “utilizador”, “cidade” ou “ponto_de_interesse”. Isto deve-se ao facto de a tabela de avaliações referenciar, através de chaves estrangeiras (FOREIGN KEY), as entidades que estão a ser avaliadas e o autor da avaliação. Uma inversão nesta ordem conduziria inevitavelmente a uma violação da integridade referencial, rejeitada pelo SGBD.

Para solucionar este problema e determinar a sequência correta de inserção, a equipa recorreu à ordenação das dependências do esquema relacional.

Analisando as dependências, definiu-se a seguinte ordem de povoamento:

1. Entidades Independentes: “utilizador” e “cidade” (não dependem de chaves estrangeiras externas);
2. Entidades Dependentes de 1º Nível: “ponto_de_interesse” (depende de “cidade”);
3. Entidades Dependentes de 2º Nível: “avaliacao” (depende de “utilizador”, “cidade” e “ponto_de_interesse”).

Implementação via Linguagem de Manipulação de Dados (SQL)

O método principal implementado pela equipa consiste no uso da Linguagem de Manipulação de Dados (LMD) do SQL, especificamente através de instruções INSERT INTO, reunidas num *script* de povoamento robusto. Este método permite um controle granular sobre os tipos de dados inseridos e garante que o ambiente de teste é determinista e reproduzível.

Para permitir a re-execução do *script* sem erros de duplicidade, implementou-se uma limpeza prévia das tabelas. Desativando temporariamente a integridade referencial (SET FOREIGN_KEY_CHECKS = 0), utiliza-se o comando TRUNCATE TABLE para remover todos os registos e reiniciar a contagem dos identificadores (AUTO_INCREMENT). Concluída a limpeza, a verificação de chaves estrangeiras é imediatamente reativada.

Considerando a estrutura da tabela “cidade”, a inserção de registos segue a sintaxe padrão INSERT INTO

Tabela(Atributos) VALUES (Valores), conforme exemplificado abaixo:

```
INSERT INTO cidade (nome, pais, classificacao_geral, classificacao_pontos_turisticos,
classificacao_gastronomia, classificacao_paisagens, classificacao_alojamentos)
VALUES
('Lisboa', 'Portugal', 4.5, 4.8, 4.7, 4.6, 4.2),
('Porto', 'Portugal', 4.7, 4.6, 4.9, 4.5, 4.4),
('Paris', 'França', 4.4, 4.9, 4.5, 4.3, 3.9),
('Londres', 'Reino Unido', 4.3, 4.8, 4.0, 4.1, 3.8),
('Roma', 'Itália', 4.6, 5.0, 4.9, 4.7, 4.0),
('Nova Iorque', 'EUA', 4.2, 4.7, 4.6, 3.8, 3.5);
```

Figura 15 - Povoamento da cidade

Na construção destas instruções, foram observadas as regras sintáticas do MySQL (Oracle, 2024). Os literais de dados textuais (*strings*), como 'Lisboa' ou 'Portugal', encontram-se delimitados por plicas (aspas simples). Já os valores numéricos, como as classificações decimais, são inseridos diretamente sem delimitadores, utilizando o ponto (.) como separador decimal para garantir a correta interpretação pelo tipo de dados DECIMAL(3,1).

Um aspecto particular desta implementação é o tratamento de valores nulos e datas na tabela “avaliacao”. As datas são passadas no formato ISO (‘AAAA-MM-DD HH:MM:SS’), e o uso da palavra reservada NULL é essencial para manter a consistência lógica da regra de negócio que dita que uma avaliação ou é sobre uma cidade, ou sobre um ponto de interesse, mas não ambos em simultâneo (ou nenhum).

```
INSERT INTO avaliacao (data_avaliacao, comentario, nota_geral, nota_pontos_turisticos, nota_gastronomia, nota_paisagens,
| nota_alojamentos, utilizador_id, cidade_id, ponto_interesse_id) VALUES
('2023-10-01 10:00:00', 'Lisboa tem uma luz incrível.', 5, 5, 4, 5, 4, 1, 1, NULL),
('2023-10-02 11:00:00', 'Bons pastéis de nata, mas muito trânsito.', 4, 4, 5, 3, 3, 2, 1, NULL),
('2023-10-03 12:00:00', 'Os elétricos são giros mas cheios.', 3, 4, 4, 4, 3, 3, 1, NULL),
('2023-10-04 13:00:00', 'Alojamento demasiado caro no centro.', 2, 5, 4, 4, 1, 4, 1, NULL),
('2023-10-05 14:00:00', 'A cidade das 7 colinas cansa as pernas!', 4, 5, 4, 5, 3, 5, 1, NULL),
```

Figura 16 - Povoamento da avaliação

Como se observa no exemplo acima, quando a avaliação se refere a uma cidade (ID 1), o campo ponto_interesse_id é explicitamente preenchido com NULL. Inversamente, quando se refere a um ponto de interesse (ID 13), o campo cidade_id mantém a referência da cidade pai, mas a lógica de negócio foca-se no ponto específico.

O ficheiro de povoamento completo, contendo os 15 utilizadores, 6 cidades, 30 pontos de interesse e 180 avaliações, encontra-se disponível no anexo "Script de Povoamento SQL". Este volume de dados foi dimensionado para permitir uma testagem eficaz das queries de álgebra relacional e dos procedimentos armazenados de ranking e listagem.

5.4. Cálculo do espaço da base de dados

É crucial calcular o espaço ocupado pelos dados na implementação de uma BD, pelo que a equipa fez uma estimativa do número registos de cada tabela, que a plataforma teria após a melhoria do sistema, e calculou o espaço ocupado por cada tabela. Por fim, também foi feita uma estimativa do crescimento do espaço necessário para guardar todos os dados.

Para calcular o espaço que a base de dados ocupa, precisamos primeiro de saber quantos bytes ocupa cada tipo de dados. Para este projeto apenas foram usados quatro tipos de dados diferentes:

- VARCHAR(N);
- INT;
- DATE.
- DECIMAL(3,2)

Os tamanhos em bytes para cada um destes tipos estão indicados na seguinte tabela:

Tipo	Tamanho(bytes)
VARCHAR(N)	1 + N, se $N \leq 64$ 2 + N, se $N > 64$
INT	4
DATE	3
DECIMAL(3,2)	2

Tabela 12- tamanho em bytes dos tipos de dados utilizados

Para cada tabela da base de dados calculámos o tamanho de cada atributo, por exemplo, na tabela utilizador precisamos do tamanho do “ID”, do “Nome”, do “Apelido”, e dos “Contactos”. Para os tamanhos de VARCHAR(N) utilizamos um valor médio estimado de caracteres para o atributo, assim temos uma estimativa mais próxima do valor real em vez de um valor máximo.

Atributo	Domínio	Valor estimado	Tamanho(bytes)
ID	INT	-	4
Nome	VARCHAR(45)	7	8
Apelido	VARCHAR(45)	8	9
Email	VARCHAR(100)	30	32
Telemóvel	VARCHAR(17)	14	15
-	-	-	Total: 68

Tabela 13- Tamanho estimado em bytes de um Utilizador

Atributo	Domínio	Valor estimado	Tamanho(bytes)
ID	INT	-	4
Data	DATE	-	3
Comentário	VARCHAR(500)	150	152
Classificação_Geral	INT	-	4
Classificação_Alojamento	INT	-	4
Classificação_Paisagem	INT	-	4
Classificação_Gastronomia	INT	-	4
Classificação_Pontos_turísticos	INT	-	4
ID_Utilizador	INT	-	4
ID_Cidade	INT	-	4
ID_Ponto_de_Interesse	INT	-	4
-	-	-	Total: 191

Tabela 14- Tamanho estimado em bytes de uma Avaliação

Atributo	Domínio	Valor estimado	Tamanho(bytes)
ID	INT	-	4
Nome	VARCHAR(45)	12	13
País	VARCHAR(45)	8	9
Classificação_Geral	DECIMAL(3,2)	-	2
Classificação_Alojamento	DECIMAL(3,2)	-	2
Classificação_Paisagem	DECIMAL(3,2)	-	2
Classificação_Gastronomia	DECIMAL(3,2)	-	2
Classificação_Pontos_turísticos	DECIMAL(3,2)	-	2
-	-	-	Total: 36

Tabela 15- Tamanho estimado em bytes de uma Cidade

Atributo	Domínio	Valor estimado	Tamanho(bytes)
ID	INT	-	4
Nome	VARCHAR(45)	12	13
Rua	VARCHAR(45)	8	9
Classificação_Média	DECIMAL(3,2)	-	2
ID_Cidade	INT	-	4
-	-	-	Total: 32

Tabela 16- Tamanho estimado em bytes de um Ponto de Interesse

Para calcular o tamanho aproximado da base de dados, a equipa precisou então de multiplicar o tamanho estimado de um registo de cada tabela pelo número de ocorrências e no final somar todos os valores obtidos.

Tabela	Tamanho estimado por ocorrência (bytes)	Número de ocorrências	Tamanho total
Utilizador	68	300	20,4 KB
Avaliação	191	200	38,2 KB
Cidade	36	500	18 KB
Ponto_de_Interesse	32	15 000	480 KB
-	-	-	Total: 556.6 KB

Tabela 17- Tamanho estimado da base de dados

5.5. Definição e Caracterização de Vistas de Utilização em SQL

O uso de vistas (*Views*) em SQL desempenha um papel fundamental na arquitetura de sistemas de bases de dados, servindo simultaneamente como mecanismo de segurança e de abstração. Uma vista comporta-se como uma tabela virtual, cujos dados não são armazenados fisicamente, mas sim derivados dinamicamente a partir de uma consulta SQL sobre tabelas base ou outras vistas.

Esta funcionalidade permite implementar uma granularidade de acesso superior à concedida pelos simples privilégios de tabela. Por exemplo, enquanto um `GRANT SELECT` na tabela “utilizador” daria acesso a todos os dados (incluindo dados sensíveis como o email ou telemóvel), uma vista permite projetar apenas a informação pública (como o nome), ocultando os restantes atributos.

No contexto do projeto “trabalhopratico”, a equipa de desenvolvimento identificou necessidades específicas que justificaram a implementação de vistas, complementando o sistema de controlo de acessos baseado em papéis definidos anteriormente. Ao contrário da abordagem de conceder acesso direto de leitura a todas as tabelas, optou-se por criar camadas de abstração para cada perfil de utilizador, simplificando a complexidade das junções (*JOINS*) e agregando dados estatísticos.

Um exemplo prático desta implementação é a necessidade do Moderador visualizar um “feed” completo de avaliações. Como as avaliações podem referir-se a uma cidade ou a um ponto de interesse (tendo chaves estrangeiras distintas), uma consulta direta exigiria lógica complexa de `JOIN` e `CASE`. Para abstrair esta complexidade, criou-se a vista `v_moderador_feed_completo`:

```
CREATE OR REPLACE VIEW v_moderador_feed_completo AS
SELECT a.id, a.data_avaliacao, u.nome AS Autor, u.email,
       CASE WHEN a.cidade_id IS NOT NULL THEN c.nome ELSE p.nome END AS Destino,
       a.nota_geral, a.comentario
FROM avaliacao a JOIN utilizador u
ON a.utilizador_id = u.id LEFT JOIN cidade c
ON a.cidade_id = c.id LEFT JOIN ponto_de_interesse p
ON a.ponto_interesse_id = p.id;
```

Figura 17 - Criação de uma vista de moderador

A diretiva CREATE OR REPLACE VIEW permite a definição desta estrutura lógica. Posteriormente, em vez de o Moderador necessitar de permissões complexas sobre múltiplas tabelas, atribui-se apenas a permissão de consulta sobre o resultado final:

```
GRANT SELECT ON v_moderador_feed_completo TO 'papel_moderador';
```

Figura 18 - Permissões para a vista de moderador

A utilização de vistas revelou-se igualmente essencial para o perfil de Gestor de Conteúdos. Os requisitos de análise implicam o cálculo de médias de classificações e contagens de pontos turísticos. Em vez de delegar estes cálculos para a aplicação cliente ou exigir que o gestor escreva consultas de agregação complexas, a equipa encapsulou esta lógica na vista `v_gestor_estatistica_cidades`. Esta vista utiliza funções de agregação como COUNT e AVG, fornecendo ao gestor dados estatísticos prontos a consumir, sem expor os dados "crus" que lhes deram origem.

Por fim, no que diz respeito ao utilizador registado, as vistas foram utilizadas primariamente como mecanismo de privacidade (Segurança ao nível da Coluna). A vista `v_utilizador_reviews_recentes` permite que um cliente consulte as opiniões de outros utilizadores sem ter acesso aos seus dados de contacto (email e telemóvel), que residem na tabela base "utilizador". Desta forma, cumpre-se o princípio do privilégio mínimo, garantindo que a informação sensível permanece protegida, mesmo quando se partilha conteúdo público.

Em conclusão, contrariamente a metodologias que delegam todo o controlo para a aplicação ou para permissões de tabela, a equipa de desenvolvimento integrou as vistas como componentes ativos da base de dados. Esta abordagem não só tornou a implementação das interrogações mais transparente para o utilizador final, como também garantiu uma camada extra de segurança e integridade, impedindo o acesso accidental a dados sensíveis ou a lógica de negócio complexa, as quais devem ser mantidas protegidas.

5.6 Tradução das interrogações do utilizador para SQL

Nesta fase do projeto a equipa escreveu scripts na linguagem de SQL para traduzir o que foi previamente escrito em álgebra relacional no ponto 4.4 deste relatório.

Primeira *query* (melhores pontos de uma cidade):

```
-- RM6: Melhores Pontos de uma Cidade
PREPARE RM6 FROM
    'SELECT
        p.nome AS Ponto,
        p.classificacao_media AS Media
    FROM ponto_de_interesse AS p
    JOIN cidade AS c ON p.cidade_id = c.id
    WHERE c.nome = ?
    ORDER BY Media DESC;';
```

Figura 19 - Query 1

Segunda *query* (listar reviews de um ponto de interesse):

```
-- RM2: Listar Reviews de um Ponto de Interesse
PREPARE RM2 FROM
    'SELECT u.nome AS Autor, a.nota_geral, a.comentario, a.data_avaliacao
    FROM avaliacao a
    JOIN utilizador u ON a.utilizador_id = u.id
    LEFT JOIN ponto_de_interesse p ON a.ponto_interesse_id = p.id
    WHERE p.nome = ?
    ORDER BY a.data_avaliacao DESC;';
```

Figura 20 - Query 2

Terceira *query* (listar reviews de uma cidade):

```
-- RM3: Listar Reviews de uma Cidade
PREPARE RM3 FROM
    'SELECT u.nome AS Autor, a.nota_geral, a.comentario, a.data_avaliacao
    FROM avaliacao a
    JOIN utilizador u ON a.utilizador_id = u.id
    LEFT JOIN cidade c ON a.cidade_id = c.id
    WHERE c.nome = ?
    ORDER BY a.data_avaliacao DESC;';
```

Figura 21 - Query 3

Quarta *query* (*Ranking* global de cidades):

```
-- RM7: Ranking Global de Cidades
SELECT
    c.nome AS Cidade,
    c.pais AS Pais,
    c.classificacao_geral AS classificacao
FROM cidade c
ORDER BY classificacao DESC;
```

Figura 22 - Query 4

Quinta *query* (*Ranking* global de pontos de interesse):

```
-- RM8: Ranking Global de Pontos de Interesse
SELECT
    p.nome AS Ponto_Interesse,
    p.rua AS Rua,
    p.classificacao_media AS Classificacao_geral
FROM ponto_de_interesse p
ORDER BY Classificacao_geral DESC;
```

Figura 23 - Query 5

5.7 Indexação do Sistema de Dados

Se corretamente utilizados, os índices são fundamentais para o desempenho de um Sistema de Gestão de Bases de Dados (SGBD). Um índice é uma estrutura de dados auxiliar (geralmente uma *B-Tree* no motor InnoDB) que organiza os valores de um ou mais atributos, permitindo ao SGBD localizar registos específicos, sem a necessidade de percorrer a tabela inteira (operação conhecida como *Full Table Scan*). Se os atributos utilizados em cláusulas de filtragem (WHERE) ou ordenação (ORDER BY) se encontrarem indexados, obtém-se geralmente um desempenho muito superior na leitura.

No entanto, a criação de índices representa um compromisso (*trade-off*): embora acelerem a leitura, ocupam espaço de armazenamento adicional e podem tornar as operações de escrita (INSERT, UPDATE, DELETE) ligeiramente mais lentas, uma vez que a árvore do índice necessita de ser atualizada a cada modificação nos dados.

Ao contrário de abordagens que adiam a indexação apenas para quando o volume de dados se torna crítico, a equipa de desenvolvimento do projeto “trabalhopratice” optou por implementar, desde a fase inicial, um conjunto de índices estratégicos. Esta decisão deve-se à natureza da aplicação (guia turístico), que é maioritariamente orientada à leitura (*read-heavy*), onde a rapidez na pesquisa de destinos e na visualização de *feeds* de avaliações é prioritária sobre a velocidade de inserção.

Em SQL, a diretiva CREATE INDEX é utilizada para criar estas estruturas. É de notar que, no motor InnoDB utilizado pelo MySQL, as Chaves Primárias e as Chaves Estrangeiras (FK) já são indexadas automaticamente para garantir a integridade referencial. Portanto, a equipa focou-se na indexação de atributos não-chave utilizados frequentemente nos Procedimentos Armazenados e nas Vistas desenvolvidos.

5.7.1. Otimização de Pesquisas por Nome (Infraestrutura)

Vários procedimentos de gestão e listagem, como o `InserirPontoInteresse` ou o `RankingPontosPorCidade`, recebem como argumento o nome de uma cidade ou ponto turístico. Para evitar que o sistema tenha de iterar por todos os registos da tabela cidade para encontrar “Lisboa”, criaram-se índices nos nomes das entidades geográficas:

```
CREATE INDEX idx_cidade_nome ON cidade(nome);
```

```
CREATE INDEX idx_ponto_nome ON ponto_de_interesse(nome);
```

Estes índices transformam a complexidade de busca linear em logarítmica, garantindo tempos de resposta imediatos mesmo que o número de cidades na base de dados cresça para a ordem dos milhares.

5.7.2. Otimização de Ordenação Cronológica (Feeds)

As vistas de utilização destinadas aos clientes e moderadores, como a `v_utilizador_reviews_recentes`,

por exemplo, apresentam as avaliações ordenadas pela data mais recente (ORDER BY data_avaliacao DESC). Sem indexação, esta operação exigiria um *filesort* (ordenação em memória ou disco) a cada execução, o que é computacionalmente dispendioso. Para mitigar este custo, criou-se um índice para esta operação e outro para outra operação idêntica que é a de organizar pelas notas:

```
CREATE INDEX idx_avaliacao_data ON avaliacao(data_avaliacao);
```

```
CREATE INDEX idx_avaliacao_nota ON avaliacao(nota_geral);
```

Desta forma, o SGBD pode recuperar os registos já ordenados diretamente da estrutura do índice, sem necessidade de processamento adicional.

5.7.3. Análise de Índices Descartados

A equipa ponderou a criação de um índice na coluna `nota_geral` da tabela `avaliacao` para auxiliar as vistas de moderação (WHERE `nota_geral` <= 2). No entanto, após análise, concluiu-se que a baixa cardinalidade deste atributo (apenas 5 valores possíveis) faria com que o otimizador do MySQL optasse frequentemente por ignorar o índice. Portanto, para poupar recursos de escrita e armazenamento, este índice específico não foi implementado na versão final.

Em conclusão, embora o volume atual de dados da BD "trabalhopratice" permitisse um desempenho aceitável sem indexação, a implementação destes índices assegura a escalabilidade futura do sistema, prevenindo a degradação de performance à medida que novos destinos e avaliações são adicionados à plataforma.

O código SQL completo para a criação destes índices encontra-se no Anexo de Otimização e Índices.

5.8 Implementação de procedimentos, funções e gatilhos

O MySQL disponibiliza um conjunto robusto de ferramentas para a definição de rotinas no lado do servidor, nomeadamente Procedimentos Armazenados (*Stored Procedures*), Instruções Preparadas (*Prepared Statements*) e Eventos Agendados (*Event Scheduler*). A utilização integrada destas funcionalidades permitiu desenvolver uma solução onde a integridade dos dados é validada na inserção e a performance de leitura é maximizada.

5.8.1 Procedimentos Armazenados e Integridade Referencial

Para operações de manipulação de dados (INSERT, DELETE) que requerem validação lógica, optou-se pela utilização de Procedimentos Armazenados. Dado que estes blocos de código contêm múltiplos terminadores de instrução (;), o comando DELIMITER foi utilizado para garantir a correta interpretação do bloco pelo compilador do MySQL.

O procedimento `InserirPontoInteresse` (RM1) exemplifica a aplicação de regras de negócio para assegurar a consistência da base de dados. Antes de efetuar a inserção de um novo ponto turístico, o procedimento executa uma verificação da existência da cidade associada:

```

DROP PROCEDURE IF EXISTS InserirPontoInteresse //
CREATE PROCEDURE InserirPontoInteresse(
    IN p_nome VARCHAR(150),
    IN p_rua VARCHAR(200),
    IN p_cidade_nome VARCHAR(100)
)
BEGIN
    DECLARE v_cidade_id INT;
    SELECT id INTO v_cidade_id FROM cidade WHERE nome = p_cidade_nome LIMIT 1;

    IF v_cidade_id IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Cidade não encontrada.';
    ELSE
        INSERT INTO ponto_de_interesse (nome, rua, cidade_id)
        VALUES (p_nome, p_rua, v_cidade_id);
    END IF;
END //

```

Figura 24 - Procedure inserir ponto de interesse

Nesta rotina, a instrução `SELECT ... INTO` tenta resolver o nome da cidade para o seu identificador. A estrutura condicional `IF ... THEN ... ELSE`, caso a cidade não seja encontrada (`IS NULL`), é levantada uma exceção personalizada através de `SIGNAL SQLSTATE`, impedindo a criação de registos órfãos e fornecendo feedback à aplicação. Uma lógica similar foi aplicada ao procedimento `CriarUtilizador`, garantindo a unicidade do endereço de email.

5.8.2. Consultas Dinâmicas e Instruções Preparadas

Para satisfazer os requisitos de listagem e filtragem de avaliações (RM2, RM3, RM5), recorreu-se a *Prepared Statements*. Esta abordagem oferece benefícios duplos:

1. Segurança: A parametrização dos inputs (através do placeholder `?`) diminui a vulnerabilidades de *SQL Injection*.
2. Eficiência: O SGBD compila o plano de execução da consulta apenas uma vez, reutilizando-o em chamadas subsequentes.

No requisito RM6 ("Melhores Pontos de uma Cidade"), a instrução preparada tira partido da otimização descrita na secção seguinte, lendo diretamente o atributo `classificacao_media` em vez de recalcular a média:

```
PREPARE RM6 FROM
'SELECT
  p.nome AS Ponto,
  p.classificacao_media AS Media
FROM ponto_de_interesse AS p
JOIN cidade AS c ON p.cidade_id = c.id
WHERE c.nome = ?
ORDER BY Media DESC;';
```

Figura 25- Query 6

5.8.3. Otimização via Eventos Agendados e Leitura Direta

Um dos desafios de performance identificados em sistemas de avaliações é o custo computacional de calcular médias (AVG) em tempo real sobre grandes volumes de dados. Para resolver esta questão nos requisitos de ranking global (RM7 e RM8), implementou-se uma arquitetura baseada em cache de base de dados atualizada periodicamente.

Foi criado o evento evt_atualizar_rankings_diario, agendado para execução automática a cada 24 horas:

```

CREATE EVENT evt_atualizar_rankings_diario
ON SCHEDULE EVERY 1 DAY
STARTS '2025-01-01 23:59:59'
DO
> BEGIN
    -- Atualizar médias das CIDADES
    UPDATE cidade c
    > INNER JOIN (
        SELECT cidade_id, AVG(nota_geral) as media_calculada
        FROM avaliacao
        WHERE cidade_id IS NOT NULL
        GROUP BY cidade_id
    - ) a ON c.id = a.cidade_id
    SET c.classificacao_geral = ROUND(a.media_calculada, 1);

    -- Atualizar médias dos PONTOS DE INTERESSE
    UPDATE ponto_de_interesse p
    > INNER JOIN (
        SELECT ponto_interesse_id, AVG(nota_geral) as media_calculada
        FROM avaliacao
        WHERE ponto_interesse_id IS NOT NULL
        GROUP BY ponto_interesse_id
    - ) a ON p.id = a.ponto_interesse_id
    SET p.classificacao_media = ROUND(a.media_calculada, 1);
- END //

```

Figura 26 - Atualização das classificações médias

Este evento realiza o processamento pesado de agregação num período de menor tráfego, atualizando os atributos estáticos `classificacao_geral` e `classificacao_media`.

Consequentemente, as consultas para os rankings globais (RM7 e RM8) deixam de necessitar de funções de agregação complexas. O requisito RM7, por exemplo, transforma-se numa operação de leitura simples e ordenada como se pode ver na figura 21 previamente mostrada.

Esta estratégia reduz drasticamente a complexidade da consulta de $O(n)$ para $O(1)$ (considerando a indexação), garantindo tempos de resposta imediatos para a funcionalidade de "Top Cidades" e "Top Pontos de Interesse" (RM8), independentemente do crescimento da tabela de avaliações.

5.8.4. Vistas (Views) para Abstração e Controlo de Acesso

Por fim, a camada de apresentação de dados foi abstraída através de Vistas (CREATE VIEW). Estas estruturas virtuais simplificam o acesso aos dados e reforçam a segurança.

A vista de gestão `v_gestor_estatistica_cidades` consolida dados de múltiplas tabelas (cidade,

ponto_de_interesse, avaliacao) utilizando LEFT JOIN para garantir a inclusão de todas as cidades no relatório, e a função COALESCE para o tratamento de valores nulos.

Em contrapartida, a vista v_utilizador_reviews_recentes foi desenhada com o princípio do menor privilégio. Ao projetar apenas o nome do autor e o conteúdo da avaliação, oculta atributos sensíveis da tabela utilizador (como email e telemóvel), permitindo que a aplicação pública consuma dados do *feed* de avaliações sem comprometer a privacidade dos utilizadores.

6. Conclusões e Trabalho Futuro

Ao longo do semestre, a equipa de trabalho desenvolveu uma solução de base de dados para a *The Traveler's Notebook*, percorrendo o ciclo completo de desenvolvimento de software, desde o levantamento e definição de requisitos, passando pela modelação concetual e lógica, até à implementação física e otimização em ambiente MySQL. Este percurso permitiu consolidar conhecimentos teóricos e adquirir competências práticas fundamentais, não apenas na linguagem SQL, mas também na arquitetura de sistemas de informação, na implementação de regras de negócio no servidor e na gestão de privilégios e de segurança.

Após a conclusão desta etapa final de implementação, o grupo ponderou sobre os resultados alcançados e a metodologia adotada. Conclui-se que os objetivos propostos foram atingidos na sua plenitude, tendo sido entregue um sistema funcional que responde a todos os Requisitos Mínimos (RM) estipulados. A base de dados permite armazenar informação, validar dados, automatizar cálculos e proteger a integridade da informação tal como a equipa pretendia.

No entanto, uma análise crítica ao processo revela aspetos que, fruto da inexperiência inicial da equipa, poderiam ter beneficiado de uma abordagem distinta. Identificaram-se lacunas na transição entre o modelo lógico e a implementação física. Várias restrições de integridade e regras de negócio (como a necessidade de validar a existência de uma cidade antes de inserir um ponto de interesse, ou a unicidade de emails) apenas foram plenamente definidas durante a codificação dos Procedimentos Armazenados. Idealmente, estas restrições deveriam ter sido detalhadas e documentadas exaustivamente nas fases de modelação anteriores. Esta "definição tardia" obrigou a um esforço acrescido de reprogramação e adaptação do esquema físico já numa fase avançada do projeto, algo que a equipa reconhece que deve ser evitado em projetos futuros através de uma especificação mais rigorosa.

Um outro ponto de aprendizagem muito importante prende-se com a otimização de desempenho. Inicialmente, a abordagem da equipa focou-se apenas na correção funcional das interrogações (obter os dados corretos). Contudo, durante a fase de testes e exploração, tornou-se evidente que certas operações de agregação originariam custos de processamento elevados em cenários de grande volume de dados.

No que toca à gestão do tempo e das tarefas, embora os prazos tenham sido cumpridos, a complexidade da programação em SQL (nomeadamente a depuração de *Stored Procedures* e a configuração de *Events*) revelou-se superior à estimada inicialmente. A curva de aprendizagem necessária para dominar a sintaxe de controlo de fluxo e a gestão de exceções no MySQL consumiu recursos temporais que, num planeamento ideal, deveriam ter sido alocados mais cedo. A equipa sentiu, em certos momentos, a dificuldade de paralelizar tarefas, uma vez que a criação de Vistas e Gatilhos dependia intrinsecamente da estabilidade das tabelas e dos procedimentos de inserção.

Em suma, o projeto resultou numa base de dados sólida, segura e otimizada, capaz de suportar as operações da agência de turismo. As dificuldades sentidas, nomeadamente na documentação de regras de negócio e na estimativa da complexidade da implementação procedimental, constituíram lições valiosas. A equipa encontra-se agora dotada de uma visão muito mais clara sobre a importância de antecipar requisitos não-funcionais, como a performance e a segurança, logo nas fases iniciais do ciclo de vida de uma base de dados.

Referências Bibliográficas

Chen, P. (1976) 'The entity-relationship model – Toward a unified view of data', *ACM Transactions on Database Systems*, 1(1), pp. 9–36.

Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377–387.

Date, C.J. (2004) *An introduction to database systems*. 8th edn. Boston, MA: Addison-Wesley.

European Union (2016) *Regulation (EU) 2016/679 (General Data Protection Regulation – GDPR)*.

MySQL AB (2024) *MySQL Reference Manual*. [online] Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/> (Consultado em: 20 de dezembro de 2025).

Oracle Corporation (2024) *MySQL Workbench Manual*. [online] Disponível em: <https://dev.mysql.com/doc/workbench/en/> (Consultado em: 20 de dezembro de 2025).

brModelo (2024) *Ferramenta de Modelação de Dados*. [online] Disponível em: <http://www.sisweb.com.br/brmodelo/> (Consultado em: 20 de dezembro de 2025).